

Лабораторная работа №30. JSON и модели данных

Цель работы:

- Разобраться с форматом JSON: структура, типы данных, вложенные объекты;
- Научиться настраивать формат JSON-ответов в ASP.NET Core;
- Применить атрибуты [JsonPropertyName] и [JsonIgnore];
- Понять как enum сериализуется в JSON и как это изменить;
- Увидеть разницу между форматом по умолчанию и настроенным.

Краткая теория (вводная часть)

Что такое JSON

JSON (JavaScript Object Notation) — текстовый формат для передачи данных. Именно в этом формате наш сервер отвечает на запросы, а фронтенд читает ответы.

Когда вы делаете GET /api/heroes — сервер не отправляет C#-объекты. Он превращает их в JSON-текст и отправляет этот текст. Браузер или JavaScript на другом конце читает этот текст и превращает обратно в объекты.

C# объект Hero —► JsonSerializer —► JSON-строка —► сеть —► браузер

{ Name = "Thor" } {"name":"Thor"}

Базовые типы в JSON:

```
{
  "string": "Тор",
  "number": 95,
  "float": 9.8,
  "bool": true,
  "null": null,
  "array": ["молния", "полёт", "сила"],
  "object": { "name": "Мьёльнир", "weight": 20 }
}
```

JSON и C#: соответствие типов

JSON	C#
"строка"	string
42	int
9.8	double
true / false	bool
null	null (для nullable-типов)
["a", "b"]	List<string>
{ "key": "value" }	Класс C#

Проблема с именами полей

В C# принято писать свойства с большой буквы: **HeroName**. В JSON принято писать с маленькой: **heroName**. Это называется **camelCase**.

System.Text.Json (используется в .NET 6+) по умолчанию сериализует свойства в camelCase. Но enum по умолчанию всё равно сериализуется как число: "universe": 0. Мы настроим явно и то, и другое — чтобы контролировать формат, а не зависеть от умолчаний.

Шаг 1. Создание проекта

1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab30_JsonModels
```

```
cd Lab30_JsonModels
```

```
mkdir img
```

4. Создайте проект **webapi**:

```
dotnet new webapi -n HeroesApi
```

5. Откройте проект в VS Code:

```
code .
```

6. Откройте терминал и выполните:

```
cd HeroesApi
```

```
dotnet restore
```

7. Установим **Swagger**. Выполните в терминале:

```
dotnet add package Swashbuckle.AspNetCore
```

```
dotnet restore
```

8. Создайте папки:

```
mkdir Models
```

```
mkdir Data
```

1.2. .editorconfig

В корне папки **Lab30_JsonModels** создайте файл **.editorconfig**. Добавьте в ваш **.editorconfig** файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

1.3. Инициализация Git

1. Выполните в терминале:

cd ..

git init -b main

cd HeroesApi

dotnet new gitignore

2. Выполните в терминале:

cd ..

touch README.md

git add .

git commit -m "Initial commit: HeroesApi project"

3. Создайте **пустой public-репозиторий** на GitHub с именем: **Lab30_JsonModels**

Не добавляйте README, .gitignore, License

4. Привяжите и отправьте:

git remote add origin <URL_репозитория>

git push -u origin main

5. Сделайте **скриншот терминала** с выполненными командами (commit, remote add, push) и сохраните в папку img с именем: **gitPushLab30_FIO.png**

Шаг 2. Модель данных

2.1. Создание модели

Создайте файл **Models/Hero.cs**, откройте и напишите код.

1. Подключаем необходимые пространства имён:

```
using System.Text.Json.Serialization;
```

Мы подключаем пространство имён **System.Text.Json.Serialization**, чтобы использовать атрибуты для управления сериализацией JSON:

- **[JsonPropertyName]** — задаёт имя поля в JSON
- **[JsonIgnore]** — исключает поле из сериализации

Без этой строки компилятор не поймёт, что означают эти атрибуты.

2. Объявляем пространство имён проекта

```
namespace HeroesApi.Models;
```

3. Создаём класс **Weapon** (Оружие):

```
public class Weapon {
    public string Name { get; set; } = string.Empty;
    public bool IsRanged { get; set; }
}
```

- `public string Name { get; set; } = string.Empty;` - свойство Name типа string с авто-реализуемыми геттером и сеттером. Инициализируем свойство пустой строкой, чтобы избежать null
- `public bool IsRanged { get; set; }` - логическое свойство: **true** — дальнобойное оружие, **false** — ближний бой

Оружие — это составная часть героя. Выделяя его в отдельный класс, мы следуем принципу **Single Responsibility** и делаем код чище и переиспользуемым.

4. Создаём перечисление **Universe** (Вселенная)

```
public enum Universe {
    Marvel,
    DC
}
```

- Marvel автоматически получает значение 0
- DC получает значение 1
- По умолчанию System.Text.Json сериализует enum как числа: 0, 1

5. Объявляем основной класс **Hero**

```
public class Hero { }
```

Далее пишем внутри класса

5.1. Уникальный идентификатор

```
public int Id { get; set; }
```

- Id — первичный ключ, уникальный номер героя в базе данных
- Тип int подходит для большинства случаев
- Сериализуется в JSON как "id": 1

5.2. Имя героя с явным указанием JSON-поля

```
[JsonPropertyName("name")]
public string Name { get; set; } = string.Empty;
```

Зачем атрибут [JsonPropertyName("name")]?

- По соглашению C# свойства пишутся с заглавной буквы: Name
- Но в JSON часто используют camelCase: name
- Этот атрибут явно указывает, что в JSON поле будет называться "name", а не "Name"

5.3. Далее объявим:

- Настоящее имя героя;
- Вселенная героя;
- Уровень силы;
- Список суперспособностей;
- Вложенный объект Weapon

```
public string RealName { get; set; } = string.Empty;
public Universe Universe { get; set; }
public int PowerLevel { get; set; }
public List<string> Powers { get; set; } = new();
public Weapon Weapon { get; set; } = new();
```

- **public Universe Universe { get; set; }** - Поле с типом-enum. В JSON выглядит как число 0, пока не настроить иначе
- **public List<string> Powers { get; set; } = new();** - Список строк. В JSON станет массивом ["молния", "полёт"]
- **public Weapon Weapon { get; set; } = new();** - Вложенный объект. В JSON станет объектом внутри объекта

5.4. Скрытое поле для внутренней логики:

```
[JsonIgnore]
public string? InternalNotes { get; set; }
```

Атрибут [JsonIgnore]:

- Полностью исключает свойство из сериализации
- Поле не попадёт в JSON-ответ клиенту

Зачем это нужно?

- Хранение служебной информации (заметки разработчика, отладочные данные)
- Защита чувствительных данных от утечки

Выполните в терминале:

git add .

git commit -m "Step 2: Hero model with JSON attributes"

git push

Шаг 3. Настройка JSON в Program.cs

Повторим, что мы писали на прошлой паре. Откроем и перепишем **Program.cs**:

```
var builder = WebApplication.CreateBuilder(args);

// Регистрация сервисов
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Middleware
if (app.Environment.IsDevelopment()) {
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Два недостатка на текущий момент:

- Имена с большой буквы: Id, RealName — не camelCase
- Enum как число: "Universe": 0 — непонятно что значит 0

Исправим оба в **Program.cs**. Подключим пространства имён в начало файла:

```
using System.Text.Json;
using System.Text.Json.Serialization;
```

И измените **AddControllers()**:

```
builder.Services.AddControllers()  
    .AddJsonOptions(options => {  
        // camelCase для всех имён: "realName" вместо "RealName"  
        options.JsonSerializerOptions.PropertyNamingPolicy =  
            JsonNamingPolicy.CamelCase;  
        // Enum как строка: "Marvel" вместо 0  
        options.JsonSerializerOptions.Converters  
            .Add(new JsonStringEnumConverter());  
        // Красивый JSON с отступами — удобно при разработке  
        options.JsonSerializerOptions.WriteIndented = true;  
    });
```

Разберём три настройки:

PropertyNamingPolicy = JsonNamingPolicy.CamelCase — все имена свойств автоматически становятся camelCase. `RealName` → `realName`, `PowerLevel` → `powerLevel`.

JsonStringEnumConverter — enum сериализуется как строка. `Universe.Marvel` → `"Marvel"` вместо 0. Это удобно — клиент сразу видит смысл значения.

WriteIndented = true — добавляет отступы и переносы строк. JSON становится читаемым в браузере и Swagger. В продакшене это обычно отключают, но при разработке очень удобно.

Практическое задание

dotnet build

Убедитесь что сборка прошла без ошибок.

Выполните в терминале:

git add .

git commit -m "Step 3: JSON configured - camelCase, enum as string"

git push

Шаг 4. Хранилище и контроллер

4.1. Хранилище в памяти

Создайте **Data/HeroesStore.cs** и вручную перепишите:

```

using HeroesApi.Models;

namespace HeroesApi.Data;

public static class HeroesStore {
    public static List<Hero> Heroes { get; } = new() {

    };
}

```

Мы снова выносим данные в отдельный класс — это чище, когда к тому же хранилищу могут обращаться несколько контроллеров

Далее пишем **внутри списка** героев `List<Hero> Heroes`:

```

public static List<Hero> Heroes { get; } = new() {
    new Hero {
        Id          = 1,
        Name         = "Человек-паук",
        RealName     = "Питер Паркер",
        Universe     = Universe.Marvel,
        PowerLevel   = 75,
        Powers       = new() { "паутина", "лазание по стенам", "паучье чутьё" },
        Weapon       = new() { Name = "Паутинострел", IsRanged = true },
        InternalNotes = "Любимый герой редактора"
    },

```

Аналогично, после запятой перепишите всех героев:

```

new Hero {
    Id          = 2,
    Name         = "Бэтмен",
    RealName     = "Брюс Уэйн",
    Universe     = Universe.DC,
    PowerLevel   = 70,
    Powers       = new() { "интеллект", "боевые искусства", "технологии" },
    Weapon       = new() { Name = "Бэталанг", IsRanged = true },
    InternalNotes = "Без суперсил, только деньги и упорство"
},
new Hero {
    Id          = 3,
    Name         = "Железный человек",
    RealName     = "Тони Старк",
    Universe     = Universe.Marvel,
    PowerLevel   = 85,
    Powers       = new() { "броня", "полёт", "интеллект", "лазеры" },
    Weapon       = new() { Name = "Костюм Марк 50", IsRanged = true },
    InternalNotes = "Я — Железный человек"
},

```



```

new Hero {
    Id          = 4,
    Name        = "Грут",
    RealName    = "Грут",
    Universe    = Universe.Marvel,
    PowerLevel  = 80,
    Powers      = new() { "регенерация", "управление деревом", "суперсила" },
    Weapon      = new() { Name = "Ветви", IsRanged = false },
    InternalNotes = "Я есть Грут"
},
new Hero {
    Id          = 5,
    Name        = "Тор",
    RealName    = "Тор Одинсон",
    Universe    = Universe.Marvel,
    PowerLevel  = 95,
    Powers      = new() { "молния", "полёт", "суперсила", "бессмертие" },
    Weapon      = new() { Name = "Мьёльнир", IsRanged = false },
    InternalNotes = "Бог грома"
},
new Hero {
    Id          = 6,
    Name        = "Росомаха",
    RealName    = "Логан",
    Universe    = Universe.Marvel,
    PowerLevel  = 85,
    Powers      = new() { "регенерация", "когти", "суперсила", "замедленное старение" },
    Weapon      = new() { Name = "Адамантиевые когти", IsRanged = false },
    InternalNotes = "Лучший у меня есть"
},
    new Hero {
        Id          = 7,
        Name        = "Дэдпул",
        RealName    = "Уэйд Уилсон",
        Universe    = Universe.Marvel,
        PowerLevel  = 80,
        Powers      = new() { "регенерация", "владение оружием", "болтовня" },
        Weapon      = new() { Name = "Катаны и пистолеты", IsRanged = true },
        InternalNotes = "Разрушает четвёртую стену"
    }
};
}

```

4.2. Контроллер

Создайте **Controllers/HeroesController.cs** и вручную перепишите.

1. Подключаем пространства имён (using)

```

using System.Text.Json;
using System.Text.Json.Serialization;
using Microsoft.AspNetCore.Mvc;
using HeroesApi.Models;
using HeroesApi.Data;

```

- **System.Text.Json** - работа с сериализацией: JsonSerializer.Serialize/Deserialize
- **System.Text.Json.Serialization** - атрибуты для настройки JSON: [JsonIgnore], JsonSerializerConverter
- **Microsoft.AspNetCore.Mvc** - базовые классы для контроллеров: ControllerBase, атрибуты [HttpGet], методы Ok(), NotFound()
- **HeroesApi.Models** - доступ к нашим моделям: Hero, Weapon, Universe
- **HeroesApi.Data** - доступ к хранилищу данных: HeroesStore

Мы подключаем **System.Text.Json** здесь, потому что в методах GetDemo и GetSerialize вручную создаём JsonSerializerOptions. В обычном контроллере без таких методов эти using не нужны

2. Объявляем пространство имён контроллера

```
namespace HeroesApi.Controllers;
```

Пространство имён HeroesApi.Controllers — это просто логическая группировка для порядка. ASP.NET Core найдёт все контроллеры автоматически через MapControllers() — вручную подключать их не нужно

3. Объявляем класс контроллера

```
[ApiController]
[Route("api/[controller]")]
public class HeroesController : ControllerBase { }
```

- **[ApiController]** - включает автоматическую валидацию моделей, автоматические ответы 400 при ошибках и другие улучшения для API
- **[Route("api/[controller]")]** - задаёт базовый URL для всех методов контроллера. [controller] автоматически подставляется имя класса без суффикса Controller → heroes

Контроллер наследуется от ControllerBase — базового класса для Web API. Он предоставляет полезные методы: Ok(), NotFound(), BadRequest() и доступ к HttpContext.

Итоговый базовый маршрут: **https://localhost:XXXX/api/heroes**

Далее пишем внутри класса.

4. Метод GetAll() — получение всех героев

```
[HttpGet]
public ActionResult<List<Hero>> GetAll() {
    return Ok(HeroesStore.Heroes);
}
```

- **[HttpGet]** - атрибут маршрутизации: метод отвечает на HTTP-запросы GET
- **ActionResult<List<Hero>>** - тип возврата: может вернуть List<Hero> или другой результат (404, 500 и т.д.)
- **HeroesStore.Heroes** - обращаемся к статическому хранилищу данных, получаем список всех героев
- **Ok(...)** - возвращает HTTP 200 OK с данными в теле ответа (автоматически сериализуется в JSON)

Запрос: GET /api/heroes

5. Метод **GetById()** — получение героя по ID

```
[HttpGet("{id}")]
public ActionResult<Hero> GetById(int id) {
    var hero = HeroesStore.Heroes.FirstOrDefault(h => h.Id == id);
    if (hero is null) {
        return NotFound(new { message = $"Герой с id={id} не найден" });
    }
    return Ok(hero);
}
```

- **[HttpGet("{id}")]** - Фигурные скобки {id} означают параметр маршрута. Значение из URL автоматически преобразуется и передаётся в параметр метода int id.
- **var hero = HeroesStore.Heroes.FirstOrDefault(h => h.Id == id);** - LINQ-метод FirstOrDefault ищет первого героя с совпадающим Id. Если не находит — возвращает null.
- **Если герой не найден:**
 - NotFound() возвращает HTTP 404
 - В теле ответа — объект с сообщением об ошибке (автоматически в JSON)
- **Успешный ответ** - возвращаем найденного героя с кодом 200 OK.

Запрос: GET /api/heroes/3

6. Метод **GetDemo()** — сравнение настроек сериализации

```
[HttpGet("demo")]
public ActionResult GetDemo() {
    var hero = HeroesStore.Heroes.First();
}
```

- `[HttpGet("demo")]` — добавляет суффикс к базовому маршруту → `/api/heroes/demo`
- `HeroesStore.Heroes.First()` — берём первого героя из списка для демонстрации

6.1. Далее добавляем **внутри метода** настройки по умолчанию:

```
var defaultOptions = new JsonSerializerOptions {
    WriteIndented = true
};
```

WriteIndented = true — красивое форматирование JSON с отступами

6.2. Добавляем внутри метода кастомные настройки:

```
var ourOptions = new JsonSerializerOptions {
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    WriteIndented = true,
    Converters = { new JsonStringEnumConverter() }
};
```

- `PropertyNamingPolicy = JsonNamingPolicy.CamelCase` - имена свойств в JSON: `name`, `powerLevel` (вместо `Name`, `PowerLevel`)
- `WriteIndented = true` - красивое форматирование с переносами и отступами
- `Converters = { new JsonStringEnumConverter() }` - enum отображается как строка: `"universe": "Marvel"` вместо `"universe": 0`

6.3. Добавляем внутри метода сериализацию + десериализацию для демонстрации:

```
return Ok(new {
    withDefaultSettings = JsonSerializer.Deserialize<object>(
        JsonSerializer.Serialize(hero, defaultOptions), defaultOptions),
    withOurSettings = JsonSerializer.Deserialize<object>(
        JsonSerializer.Serialize(hero, ourOptions), ourOptions),
    note = "Сравните имена полей и значение universe в двух вариантах"
});
```

- `JsonSerializer.Serialize(hero, options)` → Преобразует объект C# в JSON-строку с указанными настройками
- `JsonSerializer.Deserialize<object>(json, options)` → Преобразует JSON-строку обратно в объект (тип `object`, чтобы увидеть «сырую» структуру)

Маршрут **/api/heroes/demo** — это учебный инструмент. Он показывает один и тот же объект дважды с разными настройками сериализатора. Студенты сразу видят разницу между форматом по умолчанию и настроенным.

Практическое задание

Запустите сервер:

dotnet run

Откройте браузер и проверьте маршруты:

1. **http://localhost:5000/api/heroes** — список всех героев. Обратите внимание:

- Все имена полей в camelCase: `realName`, `powerLevel`
- `universe` — строка "Marvel" или "DC", не число
- Поля `internalNotes` нет — сработал `[JsonIgnore]`
- Поле `name` называется именно `name` — сработал `[JsonPropertyName("name")]`

2. **http://localhost:5000/api/heroes/1** — один герой.

3. **http://localhost:5000/api/heroes/demo** — сравните два варианта.

В `withDefaultSettings` увидите `"Universe": 0` и `"RealName"`.

В `withOurSettings` — `"universe": "Marvel"` и `"realName"`.

4. **http://localhost:5000/api/heroes/99** — должен вернуть 404 с сообщением.

Сделайте скриншот браузера с ответом `/api/heroes/1` → 

На скриншоте должно быть видно:

- **internalNotes** отсутствует
- **universe** — строка, не число
- Все поля в camelCase

Выполните в терминале:

git add .

git commit -m "Step 4: HeroesStore and HeroesController"

git push

Шаг 5. Демонстрация сериализации

Сейчас добавим в контроллер ещё один маршрут — он покажет как работает **JsonSerializer** напрямую, без контроллера.

Откройте **Controllers/HeroesController.cs** и добавьте новый метод **перед последней закрывающей скобкой**.

GET /api/heroes/serialize - Показывает как JsonSerializer работает напрямую

```
[HttpGet("serialize")]
public ActionResult GetSerialize() {
    var options = new JsonSerializerOptions { ... 1 }
    var hero = new Hero { ... 2 }
    string serialized = JsonSerializer.Serialize(hero, options);
    var deserialized = JsonSerializer.Deserialize<Hero>(serialized, options);
    return Ok(new { ... 3 }
}
```

Далее вам нужно добавить для каждой переменной (внутри скобок) код

1. Настройка options — как сериализовать:

```
var options = new JsonSerializerOptions {
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    WriteIndented = true,
    Converters = { new JsonStringEnumConverter() }
};
```

Создаёт конфигурацию для JsonSerializer:

- **CamelCase** — имена свойств в JSON будут в формате camelCase (powerLevel вместо PowerLevel)
- **WriteIndented = true** — JSON будет отформатирован с отступами (удобно для чтения)
- **JsonStringEnumConverter** — enum будет сериализоваться как строка ("Marvel"), а не число (0)

Эти настройки можно задать глобально в Program.cs, но здесь мы показываем, как управлять ими точно.

2. Создание объекта hero — что сериализуем:

```
var hero = new Hero {
    Id = 99,
    Name = "Тестовый герой",
    RealName = "Студент",
    Universe = Universe.Marvel,
    PowerLevel = 50,
    Powers = new() { "программирование", "дебаггинг" },
    Weapon = new() { Name = "Клавиатура", IsRanged = false },
    InternalNotes = "Это поле не попадёт в JSON"
};
```

Создаёт тестовый экземпляр класса Hero со всеми заполненными полями. Обратите внимание:

- **Powers** — инициализируется коллекцией строк

- **Weapon** — вложенный объект создаётся «на лету»
- **InternalNotes** — помечено атрибутом [JsonIgnore], поэтому **не должно попасть в JSON**

InternalNotes равно **null** после десериализации не потому что C# "запомнил" атрибут — а просто потому что поля **internalNotes** нет в JSON-строке (его убрал [JsonIgnore] при сериализации). Десериализатор не нашёл поле → оставил значение по умолчанию (**null**).

serialized - Сериализация: serialized — объект → JSON:

Преобразует C#-объект hero в строку JSON с учётом настроек options.

deserialized - Десериализация: deserialized — JSON → объект:

Обратное преобразование: берёт JSON-строку serialized и создаёт из неё новый объект типа Hero

3. Возвращаемый метод:

```
return Ok(new {
    serializedJson = serialized,
    deserializedObject = deserialized,
    internalNotesAfterDeserialize = deserialized?.InternalNotes ?? "null
- поле было проигнорировано"
});
```

Метод возвращает объект с тремя полями:

- **serializedJson** — «сырая» JSON-строка
- **deserializedObject** — объект, восстановленный из этой строки
- **internalNotesAfterDeserialize** — наглядная проверка, что [JsonIgnore] сработал

Практическое задание

1. Запустите сервер:

dotnet run

2. Откройте **http://localhost:5000/api/heroes/serialize**

Изучите ответ:

- **serializedJson** — строка JSON созданная из C#-объекта. Это обычный текст
- **deserializedObject** — объект восстановленный из строки. Он выглядит как обычный герой
- **internalNotesAfterDeserialize** — поле равно null потому что [JsonIgnore] не включил его в JSON, и при десериализации оно не восстановилось

Сделайте скриншот ответа /api/heroes/serialize → **img/step5_serializeLab30_FIO.png**

Выполните в терминале:

git add .

git commit -m "Step 5: serialization demo endpoint added"

git push

Самостоятельное задание

Задание 1. Фильтрация по вселенной

Добавьте к маршруту GET /api/heroes query-параметр universe:

GET /api/heroes?universe=Marvel → только герои Marvel

GET /api/heroes?universe=DC → только герои DC

GET /api/heroes → все герои

Подсказка: добавьте параметр в метод **GetAll**:

```
public ActionResult<List<Hero>> GetAll([FromQuery] string? universe = null)
```

Затем отфильтруйте список если universe не пустой.

Задание 2. Маршрут поиска

Добавьте маршрут GET /api/heroes/search?name=паук который ищет героев по имени.

Поиск должен быть регистронезависимым — паук, Паук и ПАУК дают одинаковый результат.

Подсказка: используйте `.Contains(name, StringComparison.OrdinalIgnoreCase)`.

Маршрут /search не конфликтует с /{id} — ASP.NET Core видит, что "search" — не число, и правильно выбирает нужный метод

Задание 3. README.md

Создайте README.md в корне проекта:

1. Информация о студенте — ФИО, группа, дата
2. Таблица всех маршрутов

Обязательная таблица атрибутов JSON:

Атрибут / настройка	Что делает	Пример
[JsonPropertyName("name")]	Задаёт имя поля в JSON	"name" вместо "Name"
[JsonIgnore]	Поле не попадает в JSON	Пароли, внутренние поля
PropertyNamingPolicy.CamelCase	Все поля в camelCase	"realName" вместо "RealName"
JsonStringEnumConverter	Enum как строка	"Marvel" вместо 0
WriteIndented = true	JSON с отступами	Для удобства при разработке

Главные выводы — напишите своими словами:

1. Что такое сериализация и десериализация
2. Зачем нужен [JsonIgnore] — приведите реальный пример
3. Почему enum лучше сериализовывать строкой а не числом
4. Что изменилось в ответе после настройки CamelCase

Выполните в терминале:

git add .

git commit -m "Added complete README for Lab30 and tasks"

git push

Итоговая таблица: что изучили в лабораторной

Концепция	Описание
JSON	Текстовый формат для передачи данных между сервером и клиентом
Сериализация	Объект C# → JSON-строка
Десериализация	JSON-строка → объект C#
[JsonPropertyName("name")]	Задать точное имя поля в JSON
[JsonIgnore]	Исключить поле из JSON
JsonNamingPolicy.CamelCase	Все поля автоматически в camelCase
JsonStringEnumConverter	Enum сериализуется как строка, не число
WriteIndented = true	JSON с отступами — читаемый вид
List<string>	В JSON становится массивом ["a","b","c"]
Вложенный объект	Класс внутри класса → объект внутри объекта в JSON
JsonSerializer.Serialize()	Ручная сериализация объекта в строку
JsonSerializer.Deserialize<T>()	Ручная десериализация строки в объект. <T> — тип, в который нужно преобразовать JSON. Например, Deserialize<Hero>(json) создаст объект типа Hero. <T> — это generics; тема, которую вы изучали в первом семестре